TFS: the virtual filesystem layer

TBK SPEC 11:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 11:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. The model 4

2. Mount table and dispatch 4

3. Backends and capabilities (open/closed) 5

4. COW: the composite backend (transforms law) 6

5. Encapsulation (namespace closed under mounting) 7

6. Access-mechanism matrix (by transparency) 7

7. ABI surface (additive-only rule) 7

8. libtfs for everyone 8

9. Symlink semantics (locked 2026-07-27) 8

10. Whiteout journal format (v1, locked) 9

11. The debug contract (`tebako-log`, locked) 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of the userland VFS. Status: core SHIPPED (`crates/tfs`, contract suite 164 tests); tar adapter and the COW composite (HostDir overlay + whiteout journal, mount-mode flags) SHIPPED (roadmap 12/13); the ENC transform (spec 10, `EncBackend`) SHIPPED (roadmap 37); the fd-based write family PLANNED.

1. The model

TFS is a userland VFS over bulk-storage images — the kernel-VFS shape (vfs → fs drivers → block layer) with image-in-file (or in-binary region) replacing the block device. It is **self-similar**: images are files, any file is mountable (including files inside other mounts — §5), mounts compose and stack, and each process owns its namespace object (Plan 9 per-process namespaces, delivered as a library).

Product line: **libtfs** (the engine, `tebako_fs_*` C ABI — the Rust TFS is the shipping implementation) · **tfs CLI** (human-facing: `tfs : libtfs :: sqlite3 : libsqlite3`) · **tebako-pkg** (tpkg trailer surgery — tebako's concept, not TFS's).

2. Mount table and dispatch

- N concurrent mounts: handles (`tebako_mount_t`, monotonic, never reused), each with mount point + backend instance.
- **Longest-prefix dispatch** (path-component boundary); nested mounts shadow outer ones; duplicate mount point → `EEXIST`.

© 2026 The Tebako Project

- **Coverage is not presence**: a path a mount covers but its image does not hold takes the host-passthrough decision (spec 08) — open, stat, and opendir fall through to the (policy-gated) host answer instead of surfacing the backend's ENOENT. With the app payload mounted at `/` (the v2 dispatch shape), this is what keeps the host filesystem reachable: held content serves from the image (read-only), everything else behaves as if no mount claimed it.
- fd/dir tables record the owning mount; unmount-by-handle force-closes only that mount's fds/dirs (later use → EBADF).
- Legacy `init*` single-mount semantics layered on top (`EEXIST` when anything is mounted; `unmount()` tears down everything).
- Extraction rule: 1 mount → dest root; N mounts → per-mount mount-point-basename subtrees. Extraction preserves mtime + permissions (best effort).

3. Backends and capabilities (open/closed)

Backends are pluggable drivers; new formats are additive. Capability model is honest per format — no uniform-RW pretense:

TABLE 1

format	RO	RW in-place	rebuild (creation-time)
dwarfs-t	✓	—	dwarfs-t Writer (in-process) / `mkdwarfs-t` for humans
squashfs	✓	—	mksquashfs-class
zip	✓	✓ (add/delete)	—
iso9660	✓ (PLANNED)	—	genisoimage-class
tar	✓	append-only	repack
extN	✓ (PLANNED)	✓ (PLANNED)	mkfs
FAT	✓ (PLANNED)	✓ (PLANNED)	mkfs

Per-platform backend availability (build-time, gated per-target in the consumers): dwarfs-t, zip, and the tar family build EVERYWHERE; squashfs is POSIX-only (squashfs-tools-ng is autotools/vcpkg-`!windows`), so Windows builds ship without it. A mount attempt against a gated-off backend fails with the NAMED `ENOTSUP` — never a silent re-route, never a crash.

© 2026 The Tebako Project

- Detection chain: strong magic first (zip EOCD, dwarfs, squashfs, iso9660 `CD001`@0x8001, ext `0xEF53`@0x438, FAT BPB), weak heuristics (tar) last, with per-candidate confidence; the claimed backend is reported.
- Mounts carry explicit mode flags: `TEBAKO_MOUNT_RO` (default), `_COW`, `_RW`. Writes on RO mounts → `EROFS`.

4. COW: the composite backend (transforms law)

`CowBackend { base: dyn Backend, overlay: dyn Backend }` — stacking, not a format: reads fall through to base unless shadowed; writes/deletes/attr changes land in the overlay; whiteouts (a small journal) hide base entries. Exists ONLY in the Rust TFS — dwarfs-t and all backends stay read-only (spec 00, invariant 5). SHIPPED: `CowBackend` over `HostDirBackend`, whiteouts masking base entries only (an overlay entry of the same name wins — upper-replaces-whiteout, overlayfs semantics).

- First overlay: HostDirBackend (a host directory exposed as a TFS backend — independently useful) + whiteout journal. Disposable by deleting the dir. SHIPPED (journal: `.tfs-whiteouts` inside the overlay, strict v1 text format, atomic rewrites — the delete-side audit delta).
- Stackable, swappable, self-contained: an overlay may itself be a CowBackend (layer stacks); an overlay may be a portable image file (detach, ship, re-attach); a base image may CARRY its overlay inside itself.
- Write-side lifecycle: writes target the fast store during use (no compression in the hot path); heavyweight compression is a POST-PROCESS at unmount or live: compress the overlay into a dwarfs layer (algo per content), JOIN/squash layers (resolving whiteouts), or CHECKPOINT (squash the overlay into a new RO layer beneath and reset — running snapshots). Export any layer/overlay/composite view to any output format via the Writer. The whiteout journal IS the audit delta.

© 2026 The Tebako Project

5. Encapsulation (namespace closed under mounting)

Mount-source kinds: host-file, memory, file-region (offset+length), and VFS-file-region — `tebako_fs_mount_from_vfs(vfs_path, mount_point, …)` mounts an image addressed by a path INSIDE an existing mount; the new backend reads its bytes through the owning mount's pread. "An FS includes another FS at a directory path" is a primitive; every file in the namespace is a potential filesystem.

6. Access-mechanism matrix (by transparency)

1. link — in-process via libtfs (fastest, deepest; the packaged-app model).
2. fuse — `tfs mount image /mnt --fuse` (PLANNED): real system-wide mount wherever FUSE exists; multi-backend + multi-mount (nested mount points appear as directories). The ONE mechanism with a host dependency.
3. serve — `--serve=nfs|webdav` (PLANNED): userland server; the OS's native client mounts it — no FUSE anywhere.
4. shell — interactive browse + one-shots (`tfs ls/cat/tree/stat`).
5. exec — `tfs exec image -- cmd` (SHIPPED, roadmap 30): VFS injected via the preload interposition shim (spec 07 §8, tier 1) — the MAINLINE native-exec mechanism, and a TFS consumer like any other. macOS and linux-gnu first-class; windows later.
6. extract — materialize to disk (exists; the honest fallback).

Trade-off stated plainly: without the kernel there is no system-wide transparent mount for unmodified arbitrary processes; TFS trades that for zero privileges, every platform, in-process speed.

7. ABI surface (additive-only rule)

Current: mount/multi-mount family, stat/pread/dir family (incl. `rewinddir`, `telldir/seekdir` index cookies, `dir_is_embedded`), `extract_all`, `dlmap2file` (memfs path → host cache for dlopen), `tebako_fs_host_policy` + `tebako_host_mount_t` (spec 08 jails — gates the host-passthrough path of every IO route; additive), `tebako_fs_host_`

© 2026 The Tebako Project

guard (spec 08 §3 — answers whether the installed policy judges an UNCOVERED host path, so a runtime's IO shim can widen its route predicate from coverage to "coverage, or the host namespace is guarded"; additive), abi_version() = 1, plus the additive mount-with-mode entry points (tebako_fs_mount_from_{file,file_at,memory}_with_mode taking TEBAKO_MOUNT_RO/_COW/_RW; SHIPPED — RO semantics unchanged, COW stacks the composite, RW is ENOTSUP). Write family (gated by mount mode): write/pwrite/mkdir/rmdir/unlink/rename/chmod/utimens/truncate/ fsync — ADDITIVE; RO-only consumers see zero change; abi version bumps per spec 14. Exported symbols: exactly tebako_* (nm-verified).

8. libtfs for everyone

The C ABI is the product: ONE engine (Rust), ONE machine contract, thin per-language adapters in their own ecosystems (SQLite/zlib/libgit2 model). Shared + static libs, pkg-config, embedding guide, per-backend license statement (dwarfs-t: GPL-3.0; zip: BSD; squashfs-tools-ng: LGPL-3.0 — backend selection changes a consumer's obligations; stated, never softened).

9. Symlink semantics (locked 2026-07-27)

Images contain symlinks; resolution is rule-bound:

- Relative targets resolve within their own mount (never escape the mount's tree).
- Absolute targets resolve through the ROUTER: the longest-prefix mount table — a symlink /__app__/share/x → /opt/inkscape/share/y lands in the /opt/inkscape mount when present (cross-mount links WORK — that is the point of composition).
- Targets outside every mount are host-passthrough, subject to the SAME host_policy as any host path (spec 08): denied under a deny policy (EPERM), read-only under an ro grant, open otherwise. A symlink can never widen a jail.
- Backends without symlink notions (zip-era) treat links as regular entries (stat serves the target per the stat-serves-lstat rule).

The ABI surface for links (three verbs, POSIX-shaped): lstat reports the entry itself (S_IFLNK, st_size = target length), readlink returns the stored target verbatim (relative stays relative; an undersized buffer is ERANGE, never a silent truncation), and stat follows.

© 2026 The Tebako Project

open follows unless `O_NOFOLLOW` is set; `opendir` follows a link to a directory; `access` follows. Resolution is bounded at 32 hops with `ELOOP`, in-process and per-call. The preload shim maps the libc spellings onto these verbs — `stat/stat64/__xstat` follow, `lstat/lstat64/__lxstat` and `readlink` do not.

10. Whiteout journal format (v1, locked)

The COW overlay's `.tfs-whiteouts` (spec 11 §4): strict v1 text, one record per line — `<op> <path>` where op ∈ {`rm`, `rmd`} (file, subtree); paths are %-escaped (space, newline, % itself); the journal is rewritten atomically (tmp+rename) on every mutation; ANY malformed line fails the mount with EINVAL (strict, never lenient). Whiteouts mask BASE entries only; an overlay entry of the same name always wins.

11. The debug contract (`tebako-log`, locked)

One facility (`crates/tebako-log`), every component wired through it — no ad-hoc `eprintln!` anywhere in the stack.

- `TEBAKO_DEBUG` — the level: `off` (default) / `error` / `warn` / debug / `trace`, plus per-component overrides (`debug,preload=trace,tfs=warn`). The legacy boolean TEBAKO_DEBUG_TFS maps to debug for back-compat.
- `TEBAKO_DEBUG_FILE` — the sink: stderr by default (**stdout belongs to the payload, never to the log**); a path opens append-mode with %p → pid expansion so exec'd children don't clobber one file; parents created; unopenable → stderr with one warn line.
- `TEBAKO_DEBUG_COMPONENTS` — comma filter of component names (default: all). Components are crate names: `preload`, `tfs`, `driver`, `shim`, `bootstrap`, `cli`, pkg, `resolve`.
- **Format**: one parseable line per event — `tebako[<pid>] <level> <component>: <event> <k=v>…`
- **Discipline**: zero cost when off (the config reads once, the gate runs before any formatting); paths, decisions, and digests are loggable — payload contents and key material never are.

© 2026 The Tebako Project

The write gate's discriminator is `path_is_held` (§3): a path is held when the image holds an entry at it OR at any existing in-image ancestor (implied parents count — zip has no explicit dir entries, dwarfs always does; `Backend::has_entry_or_children` answers per backend). Writes into a held tree are EROFS; covered paths with no in-image ancestor pass to the host, policy-gated (§6, spec 08).

© 2026 The Tebako Project